

Файловые системы семейства extX

Инструменты

xxd

`xxd -h[elp]`

`xxd [ключи] [входной_файл [выходной_файл]]`

`xxd -r[evert] [ключи] [входной_файл [выходной_файл]]`

Основное назначение команды `xxd` заключается в преобразовании бинарных файлов и дисковых разделов в символьный вид. Основной выходной формат представлен в виде стандартного шестнадцатеричного дампа. После преобразования мы имеем возможность работать с выходными данными как с текстом и использовать всю мощь утилит предназначенных для обработки данного текста. Использование утилиты `xxd` является обязательным при защите консоли от повреждения при выводе непечатных символов.

Утилита `xxd` предоставляет также возможность обратного преобразования текстового формата в бинарный вид. Это используется для вставки и замены байтов и символов в первоначальный массив данных.

Имеется возможность отобразить бинарный код в виде непрерывной цепочки символов в котором каждому байту будет соответствовать два символа.

xxd

Основные параметры:

-b – информация выводится в бинарном виде.

-c N – определить количество байтов <N>, которое выводится в каждой строке. Значение по умолчанию: 16. Максимально допустимое значение: 256.

-p | -ps | -postscript – использовать непрерывный формат вывода шестнадцатеричного кода, известный как «простой» стиль или стиль «postscript».

-l N – определяет количество байтов <N>, которое будет выведено на консоль.

-r – выполнить обратное преобразование.

-g байт| -groupsize – позволяет выполнять группировку указанного количества байтов (две шестнадцатеричные цифры или восемь битов), отделяя группы друг от друга пробелами.

-r | -revert – изменяет смысл операции на противоположный: позволяет выполнять преобразование шестнадцатеричного представления в бинарный код (или применять результат в качестве заплатки).

-s [+][-]seek – начинает работу с указанного абсолютного (или относительного) <смещения> в байтах во входном_файле.

-u – использует шестнадцатеричные цифры в верхнем регистре. По умолчанию используются цифры в нижнем регистре символов.

yes

Осуществляет непрерывный вывод в стандартный поток заданной строки. Если строка не указана, то будет выводиться символ «y».

dd

```
dd [--help] [--version] [if=file] [of=file] [ibs=bytes]  
[obs=bytes] [bs=bytes] [cbs=bytes] [skip=N]  
[seek=N] [count=N] [conv={...}]
```

Команда `dd` является одной из наиболее старых команд `unix/linux`. Ее отличает неповторимый синтаксис и многообразии сфер применения. Эта команда умеет копировать текстовые и бинарные массивы данных и предназначена для создания клонов электронных носителей информации и их разделов, копирования оперативной памяти, создания файлов образов ЭНИ (электронных НИ) и замены символов (цепочек байтов). В этой команде имеется множество параметров, причем значение параметру присваивается с использованием знака равенства (=).

dd

Основные параметры:

if=file – источник копирования. В качестве имени файла можно использовать как обычные абсолютные и относительные имена, так и имена устройств блочного типа (/dev/sda1, /dev/cdrom, /dev/mem)

of=file – цель или место назначения для копируемых данных. Тип файла как и в случае с параметром if.

bs (block size) – единица измерения копируемых данных. Может быть от одного байта и выше. По умолчанию, когда параметр bs опущен, используется размер одного сектора (512 байт). Величины блоков размерностью в килобайты, мегабайты, гигабайты и т.д. обозначают прописными буквами 4K, 2M, 16G. При копировании блоков размером меньше одного сектора происходит посекторное копирование, а из этого значения отделяется нужное число байт. Максимальная скорость копирования достигается при размере блока равного 4K. В этом случае обеспечивается равенство блока дисковой памяти и страницы оперативной памяти, что наиболее благоприятно в процедурах своппинга.

dd

Основные параметры:

count, **skip**, **seek** – соответственно число копируемых блоков размерностью **bs**, и величины смещения относительно начала файлов или дисковых устройств. Параметр **skip** указывается применительно к источнику копирования, а параметр **seek** – к месту назначения. Команда **dd** не распознает шестнадцатеричных чисел, поэтому, указанные величины должны вводиться в десятичном виде. Допускается использование промежуточной функции для пересчета шестнадцатеричных чисел в десятичные:

```
count=$(( 16#fe12ed45 ))
```

Перед копированием следует количественно оценить, вмещается ли копируемый участок данных в отведенное место.

status=none – при выводе данных на экран или в файл подавляется сопутствующая информация о количестве скопированных данных, результатах копирования, результатах копирования и скорости копирования.

status=progress – в ходе копирования отображается информация о скорости копирования и числе скопированных блоков. Информация динамически обновляется.

Для параметров **bs**, **ibs**, **obs** рекомендуется выставлять значения, кратные **512**. Для указания значений bytes и N допускается использовать суффиксы: c=1; w=2; b=512; kB=1000; K=1024; MB=1000*1000; M=1024*1024; GB=1000*1000*1000; G=1024*1024*1024. Значения с использованием T, P, E, Z, Y формируются по такому же принципу.

dd

Основные параметры:

Параметр **conv** используется при необходимости преобразования или перекодировки входного потока. Чаще всего используются три варианта записи:

conv=noerror — копирование продолжается до первого сбоя. Если в результате трех считываний одного и того же блока не удастся получить мажоритарного (когда из трех попыток два раза скопировано одно и тоже число) числа блок считается поврежденным и на его место записываются нули.

conv=notrunc — при копировании данных в файл благодаря этой команде исключается обрезание длины файла при завершении копирования.

conv=swab — при копировании четные и нечетные байты меняются местами. Этот режим необходим при воспроизведении данных, представленных в кодировке unicode.

debugfs

Отладчик файловой системы. Может использоваться для получения информации о структуре файловой системы. В какой-то степени она позволяет рассмотреть индексные дескрипторы неудаленных файлов и каталогов.

Базовая команда

```
debugfs -R stats /dev/sda1
```

В таком виде debugfs используется для отображения содержимого суперблока (в читаемом виде), а также, описателей групп блоков. Информация о группах блоков выводится достаточно верно и позволяет наглядно отобразить структуру и содержимое битовых карт, таблиц индексных дескрипторов и файлового журнала. Кроме того, debugfs способна отобразить в шестнадцатеричном виде и человеческой интерпретации содержимое индексных дескрипторов неудаленных файлов, а также, заполнение блоков данных.

Файлы

Типы файлов

-	Обычный файл
d	Каталог
b	Блочное устройство (оборудование, обмен данными с которым происходит на уровне блоков, например, жесткие диски.)
c	Символьное устройство (низкоуровневые узлы, которые не нуждаются в блочной передаче данных, например, клавиатура)
s	Сокет (файл, открываемый процессами при помощи специальных функций и обеспечивающий двусторонний обмен данными)
p	Именованный канал (канал FIFO, обеспечивает одностороннюю передачу данных между двумя и более процессами)
l	Символическая ссылка

Типы файлов

-	Обычный файл
d	Каталог
b	Блочное устройство (оборудование, обмен данными с которым происходит на уровне блоков, например, жесткие диски.)
c	Символьное устройство (низкоуровневые узлы, которые не нуждаются в блочной передаче данных, например, клавиатура)
s	Сокет (файл, открываемый процессами при помощи специальных функций и обеспечивающий двусторонний обмен данными)
p	Именованный канал (канал FIFO, обеспечивает одностороннюю передачу данных между двумя и более процессами)
l	Символическая ссылка

Ext2 и Ext3

Записи каталога

file1.txt

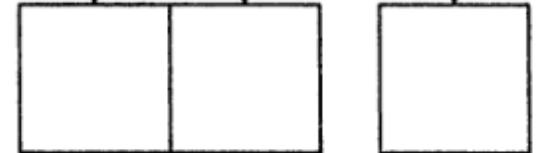
file10.txt

Индексные узлы

[Метаданные]

[Метаданные]

Блоки содержимого



Метаданные

Хранение информации

Вся информация хранится в блоках

Часть блоков используется для хранения метаданных (либо зарезервировано системой).

Остальные блоки используются для хранения содержимого файлов.

Метаданные

К метаданным относятся данные, описывающие файлы или каталоги.

Основные метаданные хранятся в структурах данных индексных узлов.

Расширенные атрибуты могут содержать дополнительные метаданные.

Основные сущности

- суперблок
- группа блоков
- гибкая группа блоков
- битовая карта блоков
- индексный дескриптор
- битовая карта индексных дескрипторов
- таблица индексных дескрипторов

Блоки

Загрузочный код

Загрузочный код может присутствовать, если ФС содержит ядро ОС (остальные ФС такой код не содержат).

Размещается в первых 1024 байтах ФС (перед суперблоком)

Управление на загрузочный код передает MBR (Master Boot Record – 0 сектор диска)

В случае, если в MBR располагается загрузчик (boot loader), который знает в каких блоках расположено ядро, то загрузочный код не используется – ядро ОС загружает MBR.

Блок

Блок – единица данных (группа смежных секторов).

Размер – 1024, 2048 или 4096 байт.



Значение определено в суперблоке

Адресация блоков начинается с 0. Блок №0 находится в первом секторе файловой системы.

Каждый блок принадлежит группе блоков (кроме зарезервированной области в начале ФС).

Состояние выделения

Состояние выделения блока определяется по битовой карте блоков.

Местоположение битовой карты определено в дескрипторе группы

Механизм создания файловых систем в Linux выбирает размер группы блоков таким образом, чтобы битовая карта блоков помещалась в одном блоке.

Таким образом, во всех файловых системах с байтовыми блоками размеров 4096 байт по умолчанию группа состоит из 32 768 блоков.

Пользователь может переопределить эти значения при создании файловой системы.

Состояние выделения

Не все выделенные блоки ассоциированы с файлами.

В системе существует множество выделенных блоков с административными данными файловой системы. Например, таблицы дескрипторов групп, битовые карты блоков и индексных узлов, а также таблицы индексных узлов.

Суперблок

Суперблок

Располагается со смещением 1024 байта от начала файловой системы.

Размер суперблока: 1024 байта

В первом блоке каждой группы располагается резервная копия суперблока (если не активирована функция `sparse_super`).

В суперблоке хранятся сведения о размерах и местонахождении важнейших структур данных.

В блоке, следующим за суперблоком, располагается таблица дескрипторов (описателей) групп.

Суперблок

Если активирована функция разреженного суперблока (`sparse_super`), то копии суперблока хранятся не во всех группах блоков. Список номеров таких групп блоков определяется как степени чисел 3, 5, 7.

1, 3, 5, 7, 9, 25, 27, 49, 81, ...

Суперблок

Размер поля, байт	Смещение от начала блока, байт		Назначение
	0	4	
4	0	0	Число индексных дескрипторов в файловой системе
4	4	4h	Число блоков в файловой системе
4	8	8h	Число зарезервированных блоков
4	12	Ch	Число свободных блоков
4	16	10h	Число свободных индексных дескрипторов
4	20	14h	Номер первого блока, содержащего данные
4	24	18h	Индикатор размера логического блока: 0 = 1 Кб; 1 = 2 Кб; 2 = 4 Кб
4	28	1Ch	Индикатор размера фрагментов (если фрагментация блоков предусмотрена)

Суперблок

4	32	20h	Число блоков в каждой группе блоков
4	36	24h	Число фрагментов в каждой группе блоков
4	40	28h	Число индексных дескрипторов в каждой группе блоков
4	44	2Ch	Время последнего монтирования файловой системы (в секундах с полуночи 1 января 1970 года)
4	48	30h	Время последней записи в файловую систему
2	52	34h	Число монтирований файловой системы. Если этот счетчик достигает значения, указанного в следующем поле, файловая система при перезапуске проверяется, а счетчик обнуляется
2	54	36h	Предельное число монтирований файловой системы
2	56	38h	«Магическое число» (0xEF53), указывающее, что файловая система принадлежит к ex2fs или ext3fs

Суперблок

Размер поля, байт	Смещение от начала блока, байт		Назначение
2	58	3Ah	Флаги, указывающие текущее состояние файловой системы (1 – проверенная файловая система, 2 – ФС содержит ошибки, 4 – обнаружены зависшие узлы)
2	60	3Ch	Реагирование на сообщение об ошибках в суперблоке (1 – продолжение работы, 2 – перемонтирование в режиме для чтения, 3 – паника системы)
2	62	3Eh	Дополнительная версия
4	64	40h	Время последней проверки файловой системы
4	68	44h	Максимальный период времени между принудительными проверками файловой системы
4	72	48h	Указание на тип операционной системы, в которой создана файловая система
4	76	4Ch	Основная версия файловой системы

Суперблок

2	80	50h	UID пользователя, которому разрешено использовать зарезервированные блоки
2	82	52h	GID группы, которой разрешено использовать зарезервированные блоки
4	84	54h	Первый незарезервированный индексный дескриптор
2	88	58h	Размер индексного дескриптора (00 80h или 0100h)
2	90	5Ah	Группа блоков, в которую входит данный суперблок (для резервных копий)
4	92	5Ch	Флаги совместимых функций
4	96	60h	Флаги несовместимых функций
4	100	64h	Флаги совместимых функций только для чтения
16	104	68h	Идентификатор файловой системы
16	120	78h	Имя тома

Суперблок

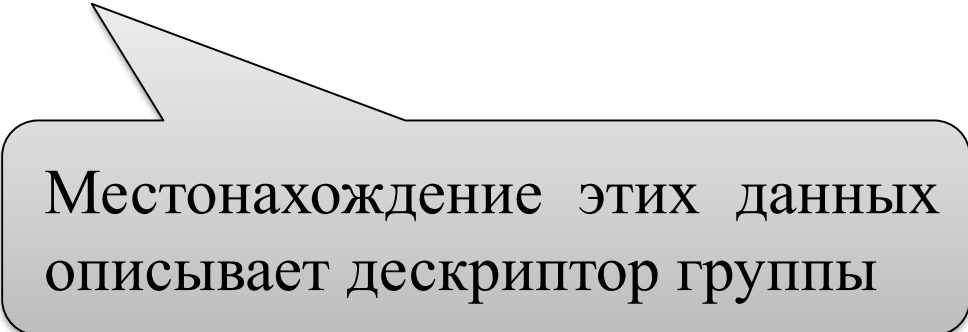
64	136	88h	Путь последнего монтирования
4	200	C8h	Битовая карта использования алгоритма
1	204	CCh	Количество блоков, предварительно выделяемых файлам
1	205	CDh	Количество блоков, предварительно выделяемых каталогам
2	206	CEh	Не используется
16	208	D0h	Идентификатор журнала
4	224	E0h	Индексный дескриптор журнала
4	228	E4h	Журнальное устройство (в случае использования внешнего журнала)
4	232	E8h	Начало списка «зависших» индексных дескрипторов
	235	ECh	Заполнение до 1024 байта

Группы блоков

Группы блоков

Содержат:

- Данные файлов;
- Административные данные:
 - резервные копии суперблока;
 - таблицы дескрипторов групп;
 - таблицы индексных узлов;
 - битовые карты индексных узлов и блоков;
 - количество свободных блоков и индексных узлов в группе блоков



Местонахождение этих данных описывает дескриптор группы

Таблица дескрипторов групп блоков (Group Descriptors Table – GDT)

Диапазон	Описание
0-3	Начальный адрес битовой карты блоков
4-7	Начальный адрес битовой карты индексных узлов
8-11	Начальный адрес таблицы индексных узлов
12-13	Количество свободных блоков в группе
14-15	Количество свободных индексных узлов в группе
16-17	Количество каталогов в группе
18-31	Не используется

Таблица дескрипторов групп

Располагается в блоке, следующем за суперблоком.

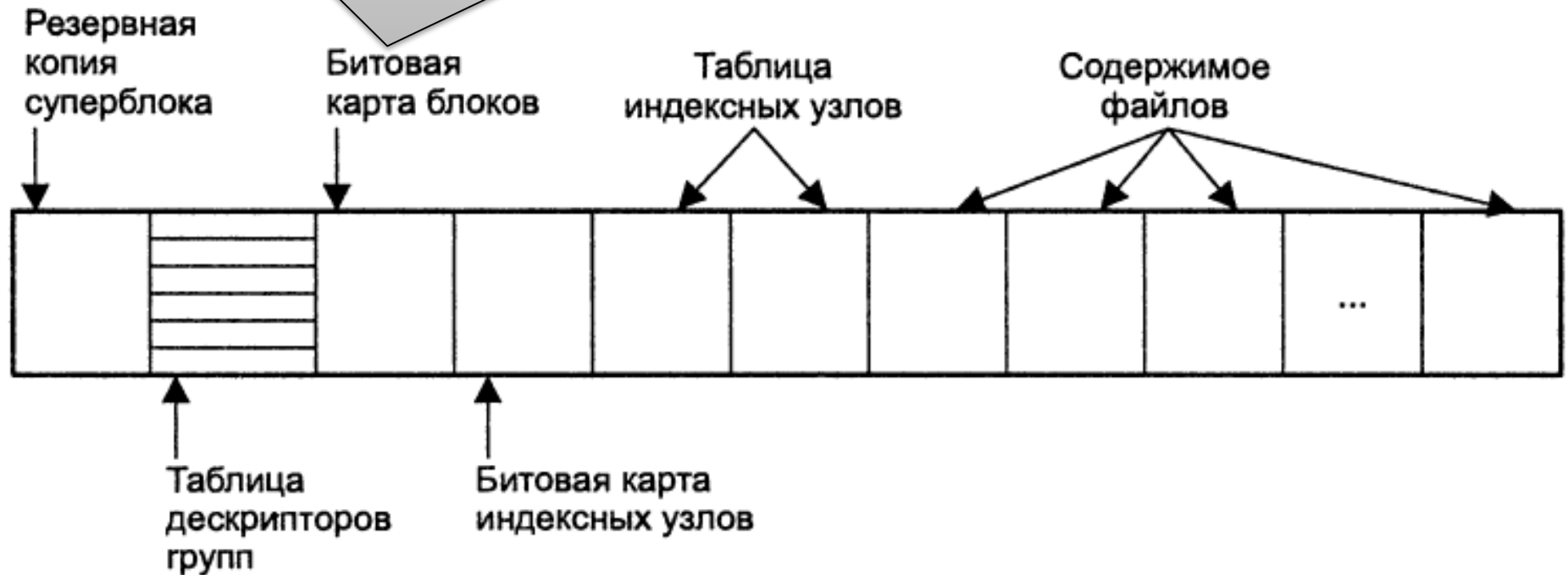
Содержит структуры данных дескрипторов групп для всех групп блоков в ФС.

Резервная копия существует в каждой группе блоков (если не активирована функция разреженного суперблока)

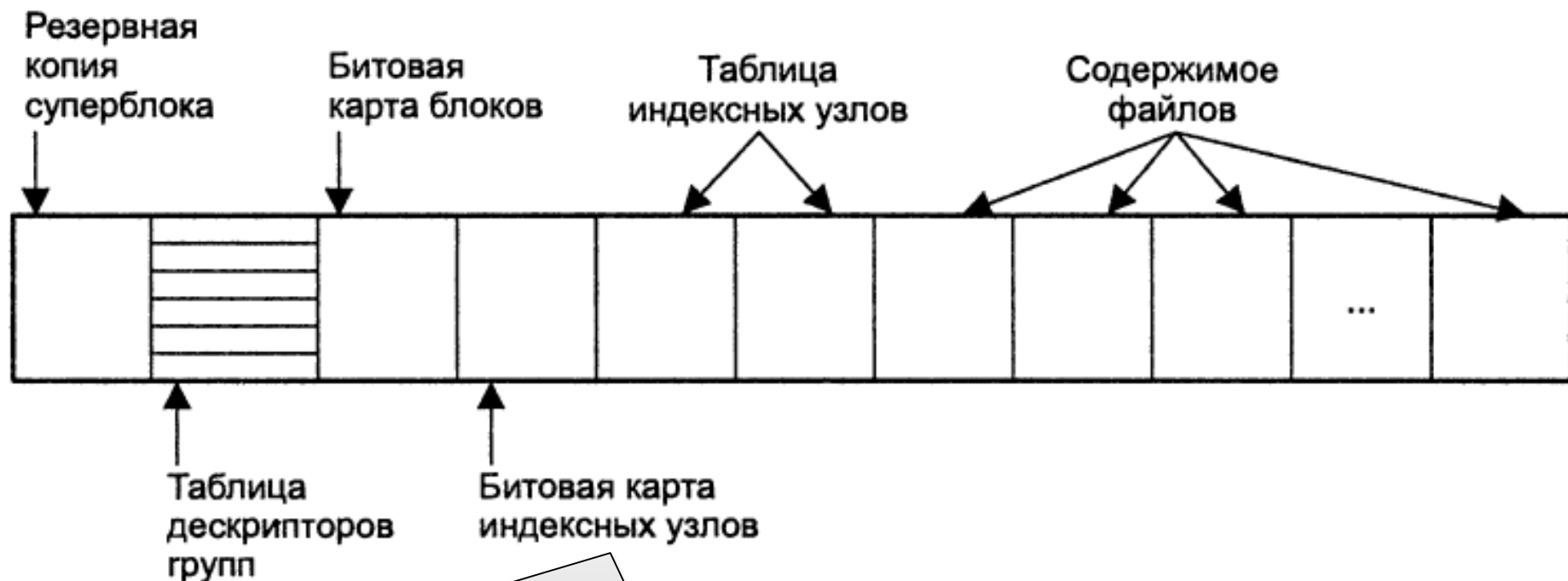
При активации функции `sparse_super` дескрипторы групп блоков располагаются только в блоках где есть копия суперблока.

Базовое строение группы блоков

Битовая карта блоков описывает состояние выделения блоков группы. Количество блоков в группе равно количеству бит в блоке. Таким образом, для хранения битовой карты блоков требуется ровно один блок.



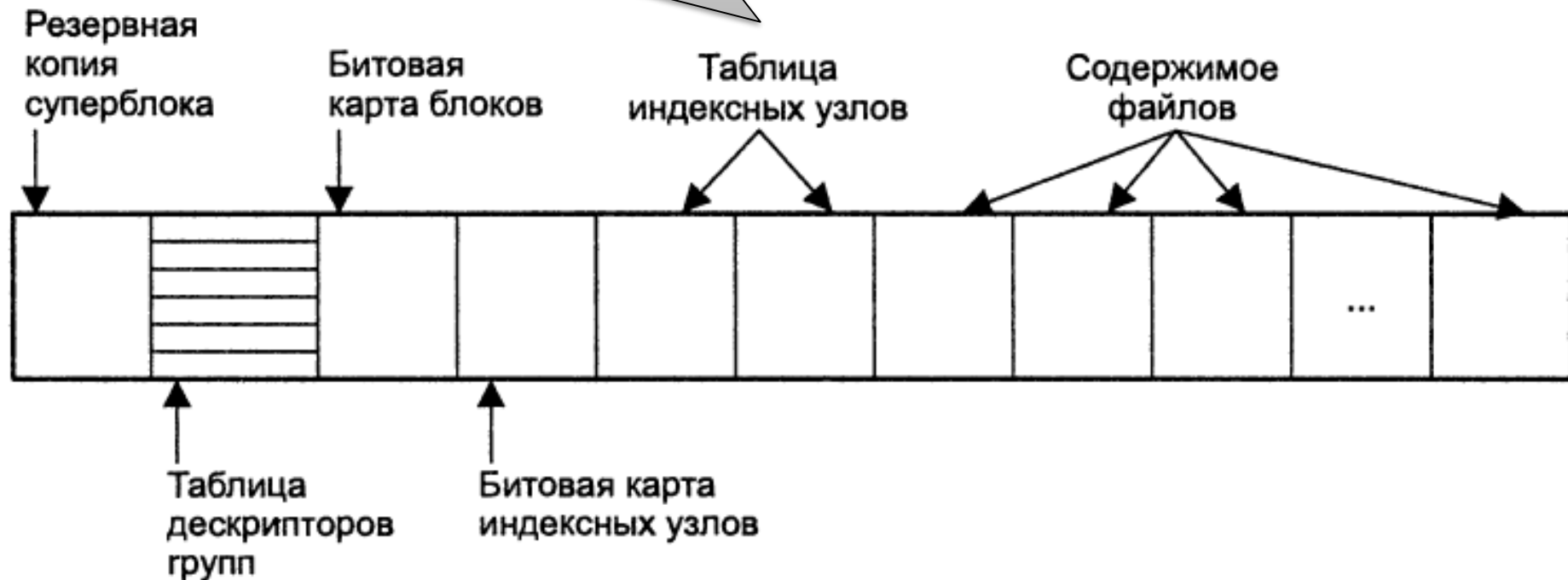
Базовое строение группы блоков



Битовая карта индексных узлов описывает состояние выделения индексных узлов в группе. Начальный адрес блока карты задается в дескрипторе группы. Как правило, количество индексных узлов в группе меньше количества блоков.

Базовое строение группы блоков

Начальный адрес блока таблицы индексных узлов содержится в дескрипторе группы, а ее размер вычисляется умножением количества индекс-индексных узлов в группе на размер каждого узла (128 байт).



Файловая система

Почему в группе блоков 32768 блока?

Под битовую карту блоков отводится один блок. При стандартном размере логического блока получаем:

$$4096 (2^{12}) \text{ байт} = 4096 * 8 = 32768 (2^{15}) \text{ бит}$$

Таким образом, битмар размером в один блок позволяет покрыть 32768 блоков

Общая организация системы

- В **ext4** вводится опция так называемых гибких групп блоков. Гибкая группа блоков состоит из нескольких групп блоков. В пределах гибкой группы битовые карты и таблицы индексных дескрипторов хранятся вместе, и лишь затем идут блоки данных.
- Это позволяет уменьшить время на чтение информации о свободных блоках и дескрипторах, а также ускорить проверку диска с помощью **e2fsck**.

Индексный дескриптор

Ext2 и Ext3

- индексные дескрипторы обладают фиксированным размером и находятся в индексной таблице
- для каждой группы блоков своя индексная таблица

Индексные узлы

Количество полей в каждом индексном узле – статическая величина.

Дополнительная информация сохраняется в расширенных атрибутах и косвенных указателях.

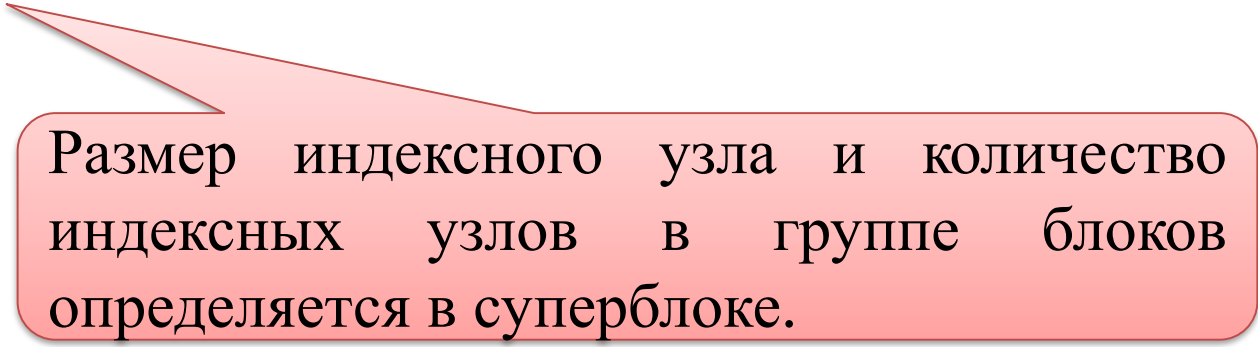
Состояние выделения индексного узла определяется по карте индексных узлов

Местонахождение карты задается в дескрипторе группы.

Индексные узлы

Все индексные узлы имеют одинаковый размер.

Каждому файлу (каталогу) выделяется один индексный узел.



Размер индексного узла и количество индексных узлов в группе блоков определяется в суперблоке.

Если адрес индексного узла известен, то его группа вычисляется по следующей формуле:

$$\text{группа} = (\text{узел} - 1) / \text{УЗЛОВ_В_ГРУППЕ}$$

Индексные узлы

Адресация индексных узлов начинается с 1. Однако, первый пользовательский файл обычно создается в узле №11; этот узел часто выделяется под каталог lost+found

Индексные узлы с 1 по 10 обычно зарезервированы.

Узел № 1 используется для хранения информации о поврежденных блоках.

Узел № 2 используется для представления корневого каталога

Узел № 2 обычно используется для журнала

Индексный дескриптор

Формат записи индексного дескриптора описан в заголовочном файле «ext4.h» как структура «ext4_inode».

```
struct ext4_inode {
    __le16    i_mode;           /* File mode */
    __le16    i_uid;           /* Low 16 bits of Owner Uid */
    __le32    i_size_lo; /* Size in bytes */
    __le32    i_atime; /* Access time */
    __le32    i_ctime; /* Inode Change time */
    __le32    i_mtime; /* Modification time */
    __le32    i_dtime; /* Deletion Time */
    __le16    i_gid;           /* Low 16 bits of Group Id */
    __le16    i_links_count; /* Links count */
    __le32    i_blocks_lo; /* Blocks count */
    __le32    i_flags; /* File flags */
    union {
        struct {
            __le32 l_i_version;
        } linux1;
        struct {
            __u32 h_i_translator;
        }
    };
};
```

...

Индексный дескриптор

Индексный дескриптор (inode) – содержит метаданные файла:

- тип файла и права доступа к нему
 - UID и GID владельца файла
 - временные метки
 - счетчик жестких ссылок
 - счетчик поколения (используется Network File System - NFS)
 - флаги файла
 - **указатели на блоки, в которых содержатся данные**
- и т.д.

	Ext3	Ext4
Размер inode	128 байт	256 байт

Индексный дескриптор

Сдвиг	Размер	Имя	Описание
0x0	__le16	i_mode	Режим файла
0x2	__le16	i_uid	Младшие 16 бит UID владельца
0x4	__le32	i_size_lo	Младшие 32 бита размера в байтах
0x8	__le32	i_atime	Временная метка доступа
0xC	__le32	i_ctime	Временная метка изменения
0x10	__le32	i_mtime	Временная метка модификации
0x14	__le32	i_dtime	Временная метка удаления
0x18	__le16	i_gid	Младшие 16 бит GID
0x1A	__le16	i_links_count	Количество жестких ссылок
0x1C	__le32	i_blocks_lo	Младшие 32 бита количества блоков
0x20	__le32	i_flags	Флаги инд. дескриптора

Индексный дескриптор

Сдвиг	Размер	Имя	Описание
0x24	4 байта	l_i_version	Версия инд. дескриптора
0x28	60 байт	i_block[EXT4_N_BLOCKS=15]	Дерево экстендов
0x64	__le32	i_generation	Полная версия
0x68	__le32	i_file_acl_lo	Младшие 32 бита блока расширенных атрибутов (ACL)
0x6C	__le32	i_size_high / i_dir_acl	Старшие 32 бита размера
0x70	__le32	i_obso_faddr	Адрес фрагмента (устаревшее поле)
0x74	__le16	l_i_blocks_high	Старшие 16 бит количества блоков
0x76	__le16	l_i_file_acl_high	Старшие 16 бит блока расширенных атрибутов
0x78	__le16	l_i_uid_high	Старшие 16 бит UID владельца
0x7A	__le16	l_i_gid_high	Старшие 16 бит GID

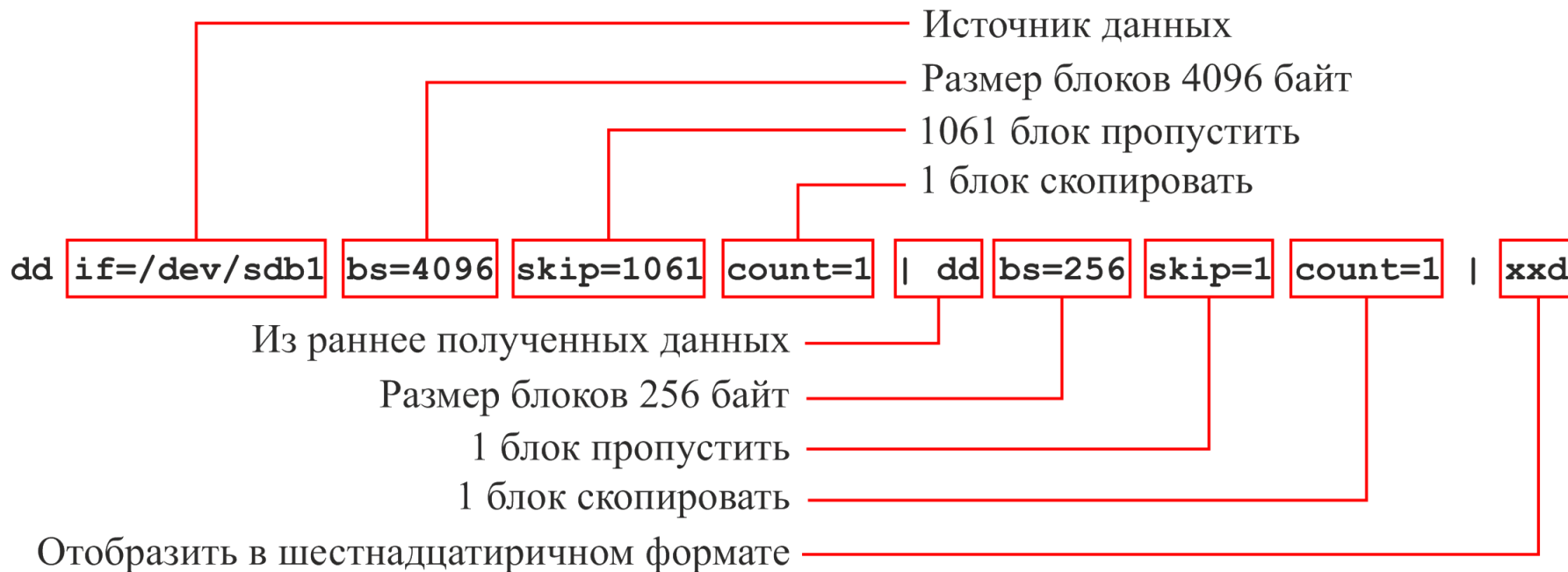
Индексный дескриптор

Сдвиг	Размер	Имя	Описание
0x7C	__le16	l_i_checksum_lo	Младшие 16 бит конт. суммы инд. дескриптора
0x7E	__le16	l_i_reserved	Не используется
0x80	__le16	i_extra_isize	Размер расширения записи инд. дескриптора
0x82	__le16	i_checksum_hi	Старшие 16 бит конт. суммы инд. дескриптора
0x84	__le32	i_ctime_extra	Наносекунды метки изменения
0x88	__le32	i_mtime_extra	Наносекунды метки модификации
0x8C	__le32	i_atime_extra	Наносекунды метки доступа
0x90	__le32	i_crtime	Временная метка создания
0x94	__le32	i_crtime_extra	Наносекунды метки создания
0x98	__le32	i_version_hi	Старшие 32 бита номера версии
0x9C	__le32	i_projid	Идентификатор проекта

Временные метки

- время последнего изменения (С-время)
соответствует последнему изменению
метаданных файла
- время последней модификации (М-время)
соответствует последней модификации
содержимого файла
- время последнего изменения (С-время)
соответствует последнему изменению
метаданных файла

Отобразить ИД корневого каталога



4096 – размер блока

1061 – начало таблицы индексных дескрипторов первой группы блоков

256 – размер индексного дескриптора

Каталоги

Каталоги

Блоки, выделенные каталогам, содержат список структур данных (записи каталогов)

Запись каталога = имя файла + адрес индексного узла + длина имени файла + адрес следующей записи каталога.

Размер каталога соответствует количеству выделенных блоков и не связан с количеством реально существующих файлов.

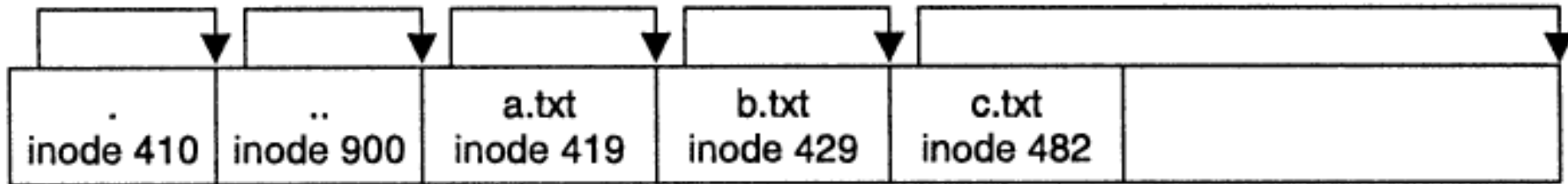
Каждый каталог начинается с записей «.» и «..», обозначающих текущий и родительский каталог соответственно. За ними следуют записи всех файлов и подката-подкаталогов в данном каталоге.

Структура файловой записи

Сдвиг	Размер	Имя	Описание
0x0	__le32	inode	Номер инд. дескриптора, на который указывает эта запись
0x4	__le16	rec_len	Длина этой записи
0x6	__u8	name_len	Длина имени файла
0x7	__u8	file_type	Тип файла: 0 – неизвестный, 1 – обычный, 2 – каталог, 3 – файл символьного устройства, 4 – файл блочного устройства, 5 – именованный канал, 6 – сокет, 7 – символическая ссылка.
0x8	char	name[EXT4_NAME_LEN]	Имя файла

Каталоги

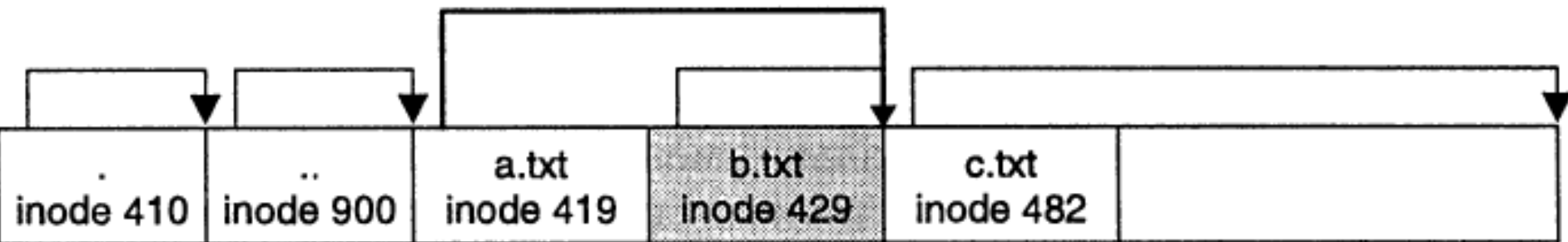
Первые две записи предназначены для каталогов «.» и «..».



Последняя запись ссылается на конец выделенного блока. Пространство за файлом c.txt не используется.

Каталоги

При удалении файла или каталога ОС скрывает запись каталога, увеличивая длину предыдущей записи, чтобы она ссылалась на запись после скрываемой.



Файл b.txt был удален, а указатель в a.txt был увеличен и переведен на c.txt. При выводе содержимого каталога запись b.txt пропускается, но данные в ней еще существуют.

Распределение данных

Корневой каталог

Debian 10.4 (46G)		
Каталог	Inode	Group
/var	131073	16
/mnt	917505	112
/srv	1179649	144
/usr	1572865	192
/usr/bin	1572866	192
/usr/sbin	1572867	192
/usr/lib	1572868	192
/root	1703937	208
/opt	1835009	224
/media	2621441	320
/etc	2883585	352

Резервирование дискового пространства

После каталогов, в которых предполагается хранение больших объемов данных (например, /var, /usr) резервируется некоторое количество дискового пространства.

Можно заметить, что во всех случаях номера групп блоков кратны 16 (16, 32, 64, 80, 112, 144, 192, 208, 224, 320, 352).

Резервирование дискового пространства

Это связано с той особенностью архитектуры ext4, что группы блоков логически группируются по 16 в так называемые гибкие группы блоков. Каждый блок с номером кратным 16 начинается с шестнадцати битовых карт блоков, за которыми идут в таком же количестве битовые карты индексных дескрипторов и таблицы индексных дескрипторов. При такой структуре файловой системы головке жесткого диска нужно совершить минимум действий, чтобы получить информацию о расположении информации на дисковом пространстве объемом 2 Гбайта ($128\text{Мбайт} \cdot 16$).

Принципы распределения информации

При распределении информации на носителе система использует два принципа. Их цель — снизить «стоимость дисковых операций»

Первый — каталог должен располагаться в непосредственной близости от своих файлов.

Второй — избегать фрагментации файлов.

Жесткая ссылка

Ссылки и точки монтирования

ФС extX реализована поддержка жестких и символических ссылок.

Жесткая ссылка — дополнительное имя файла, принадлежащего той же файловой системе.

Символические ссылки — определяют альтернативные имена для файлов и каталогов, которые могут выходить за границы файловых систем.

Жесткая ссылка

При создании жесткой ссылки:

- создается новая запись каталога, в которую помещается указатель на исходный индексный узел;
- счетчик ссылок в индексном узле увеличивается на единицу.

Теперь отличить жесткую ссылку от исходного имени
НЕВОЗМОЖНО!

Файл будет удален физически только после удаления всех жестких ссылок.

Жесткая ссылка

При создании жесткой ссылки:

- создается новая запись каталога, в которую помещается указатель на исходный индексный узел;
- счетчик ссылок в индексном узле увеличивается на единицу.

Теперь отличить жесткую ссылку от исходного имени
невозможно!

Файл будет удален физически только после удаления всех жестких ссылок.

Символическая ссылка

Символическая ссылка

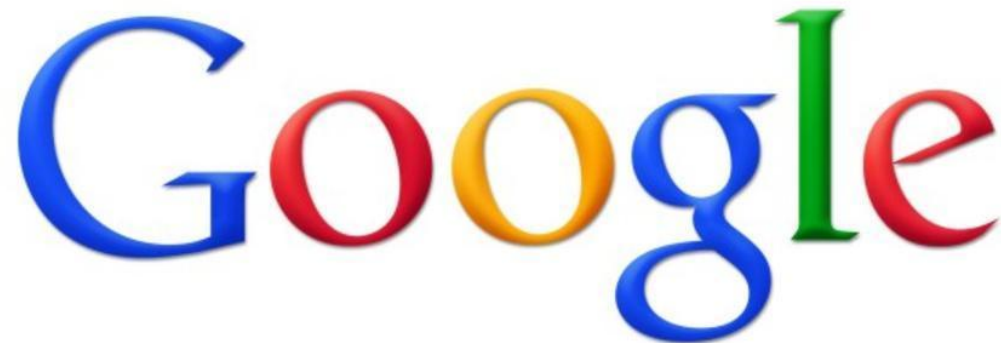
Символическая ссылка представляет собой отдельный тип файлов.



Отличия ext4

ext4

- 15 января 2010 года компания Google объявила о переводе своих серверов с файловой системы **ext2** на файловую систему **ext4**



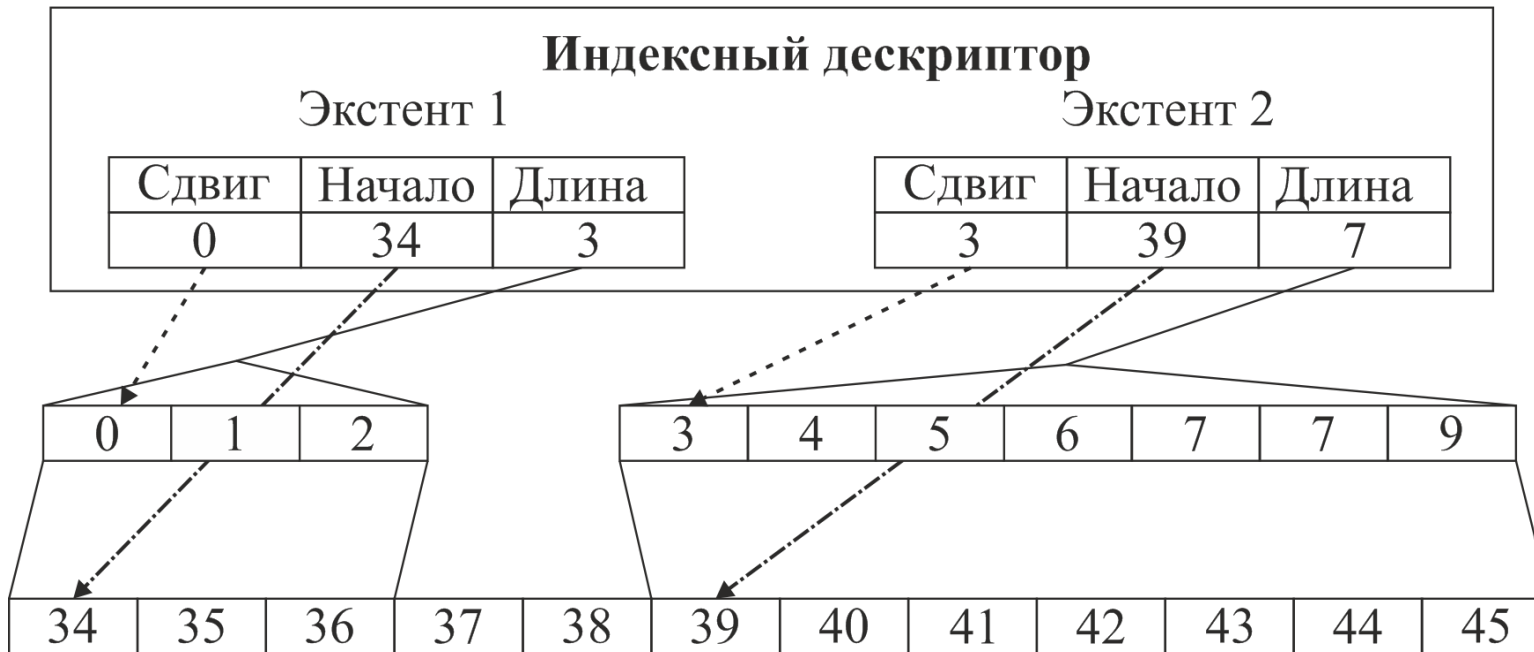
48-битные номера блоков

- **Ext3** является 32-битной файловой системой. Это значит, что ее максимальный размер при размере блока в 4 Кб составляет
- **Ext4** при размере блока 4 Кб позволяет адресовать до одного экзабайта:

$$2^{(32+12)} = 17$$

$$2^{48} \cdot 4 KB = 2^{50} \cdot 1 KB = 2^{60} B = 1 EB$$

Экстенты



Логические
блоки файла

Физические
блоки
диска

Новые алгоритмы выделения блоков и индексных дескрипторов

- **Ext3** одинаково выделяла блоки как для маленьких, так и для больших файлов, что увеличивало фрагментированность ФС.
- **Ext4** в зависимости от размера файла применяет разные алгоритмы выделения дискового пространства. Маленькие файлы из одной директории ФС старается держать рядом, чтобы минимизировать время и количество обращений к диску.

Новые алгоритмы выделения блоков и индексных дескрипторов

- *Отложенное выделение блоков* (delayed allocation). Выделение блоков для хранения данных файла происходит непосредственно перед физической записью на диск

Новые алгоритмы выделения блоков и индексных дескрипторов

- *Предварительное выделение* (persistent preallocation). В **ext4** появилась возможность зарезервировать множество блоков для записи и не тратить на инициализацию лишнее время. **Нельзя читать удаленные данные.**

Новые алгоритмы выделения блоков и индексных дескрипторов

- *Резервирование индексных дескрипторов при создании каталога (directory inodes reservation).*

Формат индексного дескриптора

- *Размер индексного дескриптора.* Размер индексного дескриптора (по умолчанию) увеличен с 128 до 256 байтов.
- *Версия индексного дескриптора.* Это может использоваться, например, для того чтобы узнавать, изменился ли файл.
- *Хранение расширенных атрибутов в индексном дескрипторе (EA in inode).*
Хранение расширенных атрибутов, таких как ACL, атрибутов SELinux и прочих, позволяет повысить производительность.

Временные отметки с наносекундной точностью

- если раньше верхней границей хранимого времени было 18 января 2038 года, то теперь – это 25 апреля 2514 года

Источники

1. Защитные механизмы операционной системы Linux: учебное пособие / В.В. Бакланов; под ред. Н.А. Гайдамакина. Екатеринбург: УрФУ, 2011. 354 с.
2. Кэрриэ Б. Криминалистический анализ файловых систем. — СПб.: Питер, 2007. — 480 с.
3. Национальная библиотека им. Н. Э. Баумана
Bauman National Library